



```
def is_vowel(word, index):
    ch = word[index].lower()

    if ch in "aeiou":
        return True

    if ch == 'y':
        if index == 0:
            return False
        return not is_vowel(word, index - 1)

    return False

def measure(stem):
    m = 0
    prev_vowel = False

    for i in range(len(stem)):
        if is_vowel(stem, i):
            prev_vowel = True
        else:
            if prev_vowel:
                m += 1
            prev_vowel = False

    return m

def contains_vowel(word):
    for i in range(len(word)):
        if is_vowel(word, i):
            return True

    return False

def ends_with_double_consonant(word):
    if len(word) < 2:
        return False
```



```
last = word[-1]
second_last = word[-2]

if last != second_last:
    return False

return not is_vowel(word, len(word) - 1)
```

```
def cvc(word):
    if len(word) < 3:
        return False

    last_three = len(word) - 3

    if (not is_vowel(word, last_three)
        and is_vowel(word, last_three + 1)
        and not is_vowel(word, last_three + 2)):

        last = word[-1]

        return last not in "wxy"

    return False
```

```
# Step 1a
def step1a(word):

    if word.endswith("sses"):
        return word[:-2]

    if word.endswith("ies"):
        return word[:-2]

    if word.endswith("ss"):
        return word

    if word.endswith("s"):
        return word[:-1]
```



Step 1b

```
def step1b(word):  
  
    if word.endswith("eed"):  
  
        stem = word[:-1]  
  
        if measure(stem) > 0:  
            return stem + "ee"  
  
    if word.endswith("ed"):  
  
        stem = word[:-2]  
  
        if contains_vowel(stem):  
            word = stem  
  
    elif word.endswith("ing"):  
  
        stem = word[:-3]  
  
        if contains_vowel(stem):  
            word = stem  
  
    if word.endswith(("at", "bl", "iz")):  
        word += "e"  
  
    elif (ends_with_double_consonant(word)  
          and word[-1] not in "lsz"):  
  
        word = word[:-1]  
  
    elif measure(word) == 1 and cvc(word):  
        word += "e"  
  
    return word
```

Step 1c

```
def step1c(word):
```



```
if word.endswith( y ):
```

```
    stem = word[:-1]
```

```
    if len(stem) > 0 and is_vowel(stem, len(stem) - 1):  
        return stem + "i"
```

```
return word
```

Step 2

```
def step2(word):
```

```
    rules = [  
        ("ational", "ate"),  
        ("tional", "tion"),  
        ("enci", "ence"),  
        ("anci", "ance"),  
        ("izer", "ize"),  
        ("abli", "able"),  
        ("alli", "al"),  
        ("entli", "ent"),  
        ("eli", "e"),  
        ("ousli", "ous"),  
        ("ization", "ize"),  
        ("ation", "ate"),  
        ("ator", "ate"),  
        ("alism", "al"),  
        ("iveness", "ive"),  
        ("fulness", "ful"),  
        ("ousness", "ous"),  
        ("aliti", "al"),  
        ("iviti", "ive"),  
        ("biliti", "ble"),  
        ("logi", "log")  
    ]
```

```
for suffix, replacement in rules:
```

```
    if word.endswith(suffix):
```

```
        stem = word[:-len(suffix)]
```



```
return word
```

```
# Step 3
```

```
def step3(word):  
  
    rules = [  
        ("icate", "ic"),  
        ("ative", ""),  
        ("alize", "al"),  
        ("iciti", "ic"),  
        ("ical", "ic"),  
        ("ful", ""),  
        ("ness", "")  
    ]  
    for suffix, replacement in rules:  
        if word.endswith(suffix):  
            stem = word[:-len(suffix)]  
            if measure(word) > 0:  
                word = stem + replacement  
    return word  
def step4(word):  
  
    suffixes = [  
        "al",  
        "ance",  
        "ence",  
        "er",  
        "ic",  
        "able",  
        "ible",  
        "ant",  
        "ement",  
        "ment",  
        "ent",  
        "ion",  
        "ou",  
        "ism",  
        "ate",  
        "iti",  
        "ous",  
        "ive".
```



```
        "ive",
        "ize"
    ]
    for suffix in suffixes:
        if word.endswith(suffix):
            stem = word[:-len(suffix)]
            if suffix == "ion":
                if (measure(stem) > 1 and
                    (stem.endswith("s") or
                     stem.endswith("t"))):
                    return stem
            else:
                if measure(stem) > 1:
                    return stem
    return word
def step5a(word):
    if word.endswith("e"):
        stem = word[:-1]
        if measure(stem) > 1:
            return stem
        if measure(stem) == 1 and not cvc(stem):
            return stem
    return word
def step5b(word):
    if measure(word) > 1 and word.endswith("ll"):
        return word[:-1]
    return word
def stem(word):
    word = word.lower()
    word = step1a(word)
    word = step1b(word)
    word = step1c(word)
    word = step2(word)
    word = step3(word)
```



```
    return word
def step5a(word):

    if word.endswith("e"):

        stem = word[:-1]

        if measure(stem) > 1:
            return stem

        if measure(stem) == 1 and not cvc(stem):
            return stem

    return word
def step5b(word):

    if measure(word) > 1 and word.endswith("ll"):
        return word[:-1]

    return word
def stem(word):

    word = word.lower()

    word = step1a(word)
    word = step1b(word)
    word = step1c(word)
    word = step2(word)
    word = step3(word)
    word = step4(word)
    word = step5a(word)
    word = step5b(word)

    return word
print("PORTER STEMMER")

word = input("Enter a Word: ")

print("Stemmed Word:", stem(word))
```

```
*** PORTER STEMMER
Enter a Word: running
Stemmed Word: run
```